Homework 09 | Question 1

SVM kernel comparison

All nine predictors are used, with famhist coded 1=Absent and 2=Present. Seed 432 assigns shared stratified five-fold splits manually. Each scaler is fitted only on the four training folds. Scores are unweighted means of held-out fold errors. All models use equal class weights and tolerance 0.001.

Kernel	C	Degree	Gamma	coef0	CV error
linear	0.1	-	-	-	28.13%
linear	1	-	-	-	28.13%
linear	10	-	-	-	27.70%
poly	0.1	2	1/9	1	27.92%
poly	1	2	1/9	1	27.05%
poly	10	2	1/9	1	27.49%
poly	0.1	3	1/9	1	26.84%
poly	1	3	1/9	1	28.79%
poly	10	3	1/9	1	33.76%
poly	0.1	4	1/9	1	26.84%
poly	1	4	1/9	1	30.08%
poly	10	4	1/9	1	35.08%
rbf	0.1	-	0.01	-	34.63%
rbf	0.1	-	0.1	-	34.63%
rbf	1	-	0.01	-	27.48%
rbf	1	-	0.1	-	29.01%
rbf	10	-	0.01	-	27.06%
rbf	10	-	0.1	-	30.52%
rbf	100	-	0.01	-	27.27%
rbf	100	-	0.1	-	35.29%

Cubic and quartic kernels with C=0.1 tie at 26.84%; prefer cubic because its degree is lower. Polynomial gamma=1/9 and coef0=1 are fixed. The best linear, quadratic, and radial scores are 27.70%, 27.05%, and 27.06%. These small differences do not demonstrate a clear advantage beyond these folds. Tuning error is not independent test error.

Question 1 (continued)

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

heart = pd.read_csv('saheart.csv')
heart['famhist'] = heart['famhist'].map({'Absent': 1, 'Present': 2})
X = heart.drop(columns='chd').to_numpy(dtype=float)
y = heart['chd'].to_numpy()
assert X.shape[1] == 9 and np.isfinite(X).all()
assert set(np.unique(y)) == {0, 1}

# Assign each class to five folds by shuffling and cycling through fold IDs.
rng = np.random.default_rng(432)
fold_id = np.empty(len(y), dtype=int)
for label in np.unique(y):
    indices = rng.permutation(np.flatnonzero(y == label))
    fold_id[indices] = np.arange(len(indices)) % 5

# 3 linear + 9 polynomial + 8 radial = 20 candidates.
candidates = []
for cost in [0.1, 1, 10]:
    candidates.append(dict(kernel='linear', C=cost))
for degree in [2, 3, 4]:
    for cost in [0.1, 1, 10]:
        candidates.append(dict(kernel='poly', C=cost, degree=degree,
                               gamma=1 / X.shape[1], coef0=1))
for cost in [0.1, 1, 10, 100]:
    for gamma in [0.01, 0.1]:
        candidates.append(dict(kernel='rbf', C=cost, gamma=gamma))
assert len(candidates) == 20

# Fit each scaler only to the four training folds; reuse these splits.
splits = []
for fold in range(5):
    train, valid = fold_id != fold, fold_id == fold
    scaler = StandardScaler().fit(X[train])
    splits.append((scaler.transform(X[train]), y[train],
                   scaler.transform(X[valid]), y[valid]))
```

Question 1 (continued)

```
rows = []
for settings in candidates:
    errors = []
    for X_train, y_train, X_valid, y_valid in splits:
        model = SVC(**settings, class_weight=None, shrinking=True,
                    tol=0.001, max_iter=-1)
        model.fit(X_train, y_train)
        errors.append(np.mean(model.predict(X_valid) != y_valid))
    rows.append({**settings, 'mean_cv_error': np.mean(errors)})
results = pd.DataFrame(rows)
results = results[['kernel', 'C', 'degree', 'gamma', 'coef0', 'mean_cv_error']]
best_index = results['mean_cv_error'].idxmin()
print(results.to_string(index=False, na_rep='—', float_format=lambda x: f'{x:.4f}'))
print('\nLowest mean CV error:')
print(results.loc[[best_index]].to_string(index=False, na_rep='—'))
```

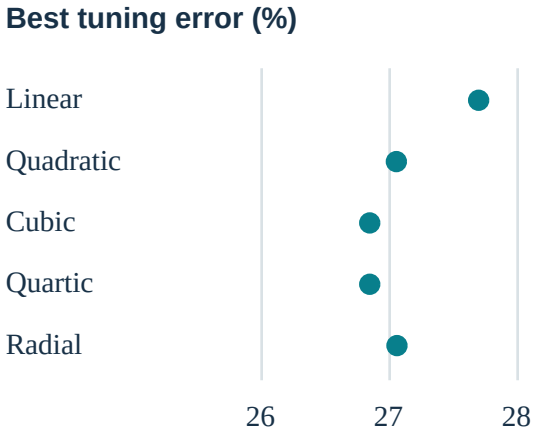
SVM analysis of coronary heart disease

Data and suitability. Predict chd for 462 observations (160 positive; 34.63%) using sbp, tobacco, ldl, adiposity, famhist, typea, obesity, alcohol, and age; famhist is 1=Absent, 2=Present. All nine predictors are numeric after coding; no missing/nonfinite values or duplicate rows occurred. Both classes support five-fold SVM fitting. No group/time identifiers are supplied; independent rows are assumed.

Design. Compare 20 settings: linear and degree-2/3/4 polynomial kernels each use C={0.1,1,10}; radial uses C={0.1,1,10,100} and gamma={0.01,0.1}. Polynomial gamma=1/9 and coef0=1. Linear is the simpler baseline; polynomial and radial allow curved boundaries. Equal class weights and overall misclassification treat both error types equally. Scaling is fitted only within training folds.

Selection and evaluation. Use shared stratified five-fold CV, minimizing mean fold error; exact ties favor simpler kernels. Nested CV uses five outer folds (seed 9432), each with five inner folds (seeds 10432-10436). Final tuning uses all rows with seed 432. The table and figure summarize final tuning, not independent test performance; all 20 settings are saved separately.

Kernel	Best C	CV error
Linear	10	27.70%
Quadratic	1	27.05%
Cubic	0.1	26.84%
Quartic	0.1	26.84%
Radial	10	27.06%



Radial best gamma=0.01; polynomial settings as above.

Final model. Cubic SVM, C=0.1, gamma=1/9, coef0=1, ties quartic at 26.84% tuning error; lower degree breaks the tie. Refit on all 462 rows yields 286 support vectors (149 negative, 137 positive). The small differences between kernels do not establish a reliable nonlinear advantage.

Internal performance. Nested CV makes 133/462 errors (28.79%), versus 160/462 (34.63%) for training-fold majority prediction. Sensitivity is 63/160 (39.38%); specificity is 266/302 (88.08%). Outer folds select four different kernel choices, indicating selection instability. Low sensitivity limits detection of positive cases.

Test status and verification. Independent test error is unavailable: these observations were already explored in Question 1 and earlier work. Nested CV estimates the selection procedure internally; it cannot undo that exploration or validate the final all-data fit externally. A new independent labeled sample with the same predictors is needed. Imports and saved-prediction metrics were verified; no fitting warnings or failures occurred.

Appendix | Complete SKILL.md

```
---
name: svm-brief
description: Fit and compare SVM classifiers and produce a one-page report only when
explicitly invoked.
---
# Scope
Apply only on explicit invocation; obtain data, response, predictors, and coding from that
invocation.
Keep dataset names, variable names, and observed results out of this skill.
# Analysis
1. Verify required imports and package versions before analysis; retain runnable code and
diagnostics.
2. Check class counts, predictor types, missing/nonfinite values, duplicates, identifiers,
leakage, and group/time dependence.
3. Judge whether sample size, class representation, and predictors support SVM classification;
 explain limitations or stop if no defensible design exists.
4. Respect supplied coding; never impute responses. Fit imputation, encoding, feature
selection, and scaling only within each training fold.
5. Audit prior access to every proposed test set before tuning. If its outcomes informed any
modeling decisions, treat it as development data.
6. Never restore independence by reshuffling previously used data. Prefer a genuinely new
independent sample for final testing when available.
7. If no untouched test set exists, state why test error is unavailable and what new data are
needed; never label training or tuning error as test error.
8. In that case, use nested CV when feasible: tune within each outer training set and evaluate
 only on its held-out fold; acknowledge prior exploratory choices.
9. Respect group/time structure; otherwise use stratified folds. Set a seed, save fold
assignments, and reuse splits across candidates.
10. Include a linear baseline and justify nonlinear kernels such as polynomial or radial from
plausible boundary complexity and available sample size.
11. Define a compact grid before evaluation: vary C; tune radial gamma and polynomial degree
where relevant; state all fixed kernel settings.
12. Select by a prespecified CV metric suited to class balance and error costs; report its
aggregation and compare a simple baseline.
13. Break exact ties by simpler kernels, then lower degree, then smaller C. Never choose
candidates using outer-fold or test results.
14. Record warnings and failures; exclude candidates with any failed fold instead of averaging
 successful folds alone.
15. After evaluation, tune on all development data and refit the selected preprocessing and
SVM; record settings and support-vector counts.
16. Evaluate a locked final model once on an untouched test set if available. Save
predictions; do not retune based on test results.
17. Report error counts/denominators and rates, plus class-specific errors when imbalance
matters; distinguish tuning, nested-CV, and independent-test results.
```

```
# Report and verification
Produce at most one printed page at 11-point font; keep code and the complete appended skill
outside that limit.
Include a compact table of candidate settings and CV results; group by kernel if full results
are retained separately.
Include a labeled compact figure showing the comparison or tuning trade-off; explain what its
scores estimate.
Prioritize data suitability, preprocessing, evaluation design, final settings, numerical
evidence, interpretation, and limitations.
Do not interpret nonlinear SVMs as causal effects or assign unsupported feature importance.
Recompute metrics from saved predictions and check agreement among prose, table, and figure.
Verify imports, coding, folds, leakage prevention, fitting diagnostics, numerical results,
figure, table, and test-status disclosure.
Render and inspect the report; verify one page, 11-point text, and no clipping. Shorten
content rather than reduce font size.
List any unmet requirements and needed inputs explicitly; never present an incomplete
requirement as satisfied.
```

Using the skill

I explicitly invoked the svm-brief skill saved in homework/svm-brief/SKILL.md and provided the dataset, response, predictors, and coding separately so the skill would remain reusable.

I asked the agent to analyze saheart.csv, predicting chd from sbp, tobacco, ldl, adiposity, famhist, typea, obesity, alcohol, and age, with famhist coded as 1 for Absent and 2 for Present. I also specified that these observations had already been used in earlier analyses, so no independent test sample was available. The analysis used the same 20 settings as Question 1 and nested cross-validation for internal evaluation. I then checked the calculations and the one-page report.